

Energy-Aware Priority Scheduling in Smart Sensor Networks Using Policy-Guided Deep Reinforcement Learning

Anand Emmanuel Raju B.¹, Radhika N.¹, and Radhika G.¹

Department of Computer Science and Engineering,
Amrita School of Computing, Coimbatore,
Amrita Vishwa Vidyapeetham, India
cb.sc.p2cse25003@cb.students.amrita.edu, {n_radhika,
g_radhika}@cb.amrita.edu

Abstract. Wireless sensor networks deployed in smart energy infrastructures must handle heterogeneous traffic profiles under strict battery constraints, reliably transmitting critical protection alerts alongside continuous telemetry. This study investigates the practical efficacy of deep reinforcement learning (DRL) for queue-level packet scheduling within an ns-3 and OpenAI Gym simulation framework. We train a Double Deep Q-Network (Double-DQN) scheduler augmented with prioritized experience replay and state-dependent policy guidance designed to prevent high-priority starvation. Through 20 matched, seed-controlled evaluation trials against Strict Priority and Random baselines, we observe that while the learned policy dramatically improves upon random allocation and maintains a 95.78% high-priority delivery ratio, it does not surpass the Strict Priority heuristic in raw high-priority latency. However, the policy-guided agent achieves a 30.5% reduction in normal telemetry delay by dynamically mitigating buffer starvation. We present these findings as a reproducible, statistically grounded benchmark defining the operational trade-offs of learning-based schedulers in smart utility networks.

Keywords: Smart Energy Systems · Wireless Sensor Networks · Reinforcement Learning · ns-3 · QoS · Priority Scheduling

1 Introduction

Modern smart utility grids rely on edge-tier Wireless Sensor Networks (WSNs) to oversee distribution feeders, monitor substation health, track consumption metrics, and detect transient transmission faults. Sensor nodes operating in these environments face harsh channel dynamics, battery depletion constraints, and bursty mixed-criticality workloads. Under these operating conditions, packets generated by monitoring devices fall into distinct service tiers: time-critical emergency messages (such as breaker trip commands, fault indications, and protection triggers) and routine periodic telemetry (such as power-quality measurements and load profiling). In accordance with industrial utility communication

profiles, critical protection messages demand immediate transmission with near-zero latency, whereas routine measurements exhibit greater delay tolerance but require dependable completion to preserve grid-wide observability [4,7].

When localized packet queues become congested, the scheduling discipline executed by sensor gateways directly determines the Packet Delivery Ratio (PDR), deadline compliance, buffer-overflow loss, and battery lifetime. Traditional grid sensor deployments favor rule-based heuristics such as Strict Priority (SP) because they are computationally lightweight, deterministic, and trivial to implement on low-power microcontrollers. Under static network loads, Strict Priority guarantees rapid transmission of high-priority traffic by invariably serving urgent queues ahead of normal queues. However, smart grid communication environments are non-stationary; channel degradation, localized interference, and sudden traffic surges expose the rigidity of static heuristics. Specifically, during extended disturbance events when sensor nodes generate sustained fault bursts, Strict Priority completely starves normal-priority queues. This causes massive buffer bloat, packet drops, and extreme delays for routine monitoring packets, impairing state-estimation algorithms at the distribution control center.

Deep Reinforcement Learning (DRL) offers an adaptive alternative by formulating scheduling as a Markov Decision Process (MDP), allowing edge gateways to learn dynamic queue management policies through continuous closed-loop interaction [8]. Nevertheless, deploying model-free DRL within mission-critical infrastructure presents substantial challenges, including training divergence, sample inefficiency, and policy collapse. In multi-objective queue management, unconstrained reward functions frequently steer agents toward degenerate local optima—such as exclusively transmitting normal packets to secure frequent, short-term rewards while allowing critical fault alerts to expire.

This paper presents an empirical investigation of a policy-guided Double-DQN packet scheduler evaluated within an integrated co-simulation framework that couples PyTorch with the ns-3 discrete-event network simulator via ns3-gym [5,2]. Rather than presenting an overstated narrative of universal superiority, we investigate a concrete systems question: can a learned DRL policy reliably maintain high-priority service quality while offering stable, competitive performance against a dedicated Strict Priority baseline under seed-synchronized evaluation conditions?

The core contributions of this work are summarized as follows:

- We construct an open-source, reproducible co-simulation pipeline integrating ns-3.40 and PyTorch, complete with run-level artifact verification and explicit per-class packet tracking.
- We design a soft, state-dependent reward guidance mechanism alongside a validation-gated model selection function that explicitly tracks high-priority generation, delivery, and queue age, preventing policy collapse.
- We conduct a statistically rigorous evaluation across 20 matched, seed-controlled simulation runs comparing the proposed policy-guided DQN against Strict Priority and Random schedulers.

- We provide a transparent characterization of the governing trade-off: although Strict Priority maintains lower high-priority delay, the learned agent substantially mitigates normal-queue starvation, reducing normal packet latency by 30.5% while sustaining a 95.78% high-priority delivery ratio.

2 Related Work

Extensive research surveys highlight the expanding role of reinforcement learning across communications, spanning medium access control, dynamic spectrum allocation, adaptive routing, and packet scheduling [8,4]. While these surveys emphasize the optimization potential of model-free architectures, they consistently identify critical deployment barriers in physical networks, including excessive sample requirements, lack of formal safety bounds, and catastrophic performance degradation under out-of-distribution traffic shifts.

Within wireless sensor and cyber-physical domains, DRL has been applied to tackle stochastic channel access and energy management. Leong *et al.* demonstrated that deep reinforcement learning can schedule sensor transmissions under Markovian packet arrival and channel fade models, showing that neural policies can adapt to system uncertainties [7]. Sharma and Chauhan developed distributed reinforcement learning algorithms for sensor node scheduling, balancing coverage maintenance against battery preservation [11]. In dense wireless IoT deployments, Dutta and Biswas explored multi-agent RL formulations to resolve channel contention at the medium access control layer [3], while Barker and Swamy examined cooperative learning frameworks to optimize multi-hop data forwarding paths [1].

Specialized studies have addressed energy conservation and latency bounds through constrained RL. Mishra and Liang investigated dynamic sleep-wake scheduling using Q-learning, demonstrating extended operational longevity through opportunistic low-power state transitions [9]. Addressing deployment safety, Nagib *et al.* detailed acceleration techniques and constraint-enforcement strategies for wireless DRL implementations, highlighting the role of reward shaping in guiding exploratory updates [10]. In time-sensitive industrial settings, Kim *et al.* verified that deep learning controllers can adaptively schedule real-time flows to conform to deterministic latency bounds [6].

In contrast to studies proposing abstract algorithmic variations without run-level artifact verification, this work provides an empirical benchmark grounded in smart grid monitoring constraints. We contribute explicit instrumentation for high-priority delivery verification, validation-gated policy selection, and full transparency regarding the performance boundaries of learned schedulers relative to established heuristics.

3 System Model and Problem Formulation

3.1 Network and Traffic Model

We model an edge sensing cluster representative of a smart grid distribution substation. The network comprises 100 wireless sensor nodes randomly distributed across a $150\text{ m} \times 150\text{ m}$ field, communicating with a central sink gateway. The physical and MAC layers are implemented in ns-3.40 using the IEEE 802.15.4 standard operating in the 2.4 GHz ISM band.

Sensor nodes generate two distinct traffic classes: High-Priority (HP) packets representing critical grid disturbance alarms, fault indications, and protection signals, and Normal-Priority (NP) packets representing periodic telemetry, power-quality measurements, and environmental status updates. High-priority traffic is generated according to an unannounced, bursty Poisson process, while normal traffic follows regular periodic generation. Each sensor maintains two dedicated, finite-capacity FIFO buffers: an HP buffer of capacity $B_H = 50$ packets and an NP buffer of capacity $B_N = 100$ packets. Packets remaining in either buffer beyond a deadline threshold ($T_H^{\max} = 1.0\text{ s}$, $T_N^{\max} = 3.0\text{ s}$) are dropped as expired timeouts. The scheduling agent situated at the gateway operates at a discrete decision interval of $\Delta t = 100\text{ ms}$ over episodes spanning 5000 decision steps (500 seconds of real-time operation).

3.2 MDP Definition

The scheduling task is formulated as an infinite-horizon discounted Markov Decision Process defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. At each discrete interval t , the agent observes the network state vector $\mathbf{s}_t \in \mathcal{S}$ and executes an action $a_t \in \mathcal{A}$.

The state space is captured by a 6-dimensional normalized observation vector:

$$\mathbf{s}_t = [q_t^N, q_t^H, e_t, a_t^N, a_t^H, d_t], \quad (1)$$

where $q_t^N \in [0, 1]$ and $q_t^H \in [0, 1]$ represent the normalized buffer occupancy ratios of the normal and high-priority queues; $e_t \in [0, 1]$ denotes the normalized residual energy proxy of the gateway node; $a_t^N \in [0, 1]$ and $a_t^H \in [0, 1]$ denote the normalized ages of the head-of-line packets in each buffer relative to their expiration limits; and $d_t \in [0, 1]$ represents the normalized active node density proxy, reflecting dynamic channel contention.

The action space consists of 5 discrete operational choices:

$$\mathcal{A} = \{\text{send HP (0), send normal (1), drop normal (2), sleep (3), drop HP (4)}\}. \quad (2)$$

Actions 0 and 1 trigger transmission attempts for packets from the respective queue. Action 3 places the wireless transceiver into a low-power sleep state for the 100 ms slot, drawing nominal idle power to conserve battery reserves. Actions 2 and 4 allow active buffer management, enabling the scheduler to purge stale packets before channel access is wasted on doomed payloads.

3.3 Reward Formulation with Soft Policy Guidance

Reward formulation is critical in mixed-criticality scheduling. A naive scalar reward based solely on aggregate throughput induces severe policy collapse: because normal telemetry packets outnumber urgent fault alarms, the agent learns to ignore HP packets entirely, maximizing transmission frequency at the expense of critical alerts. Conversely, imposing rigid, unyielding penalties for any delayed HP packet forces the neural network to degenerate into an exact replica of Strict Priority, destroying all multi-objective balancing capability.

To resolve this conflict, we formulate an outcome-driven reward augmented with state-dependent soft policy guidance. The global step reward r_t is bounded via a hyperbolic tangent scaling:

$$r_t = \tanh \left(\frac{r_t^{\text{base}} + r_t^{\text{guide}}}{100} \right). \quad (3)$$

The base component reflects primary networking performance:

$$r_t^{\text{base}} = w_1 N_{\text{sent}}^H + w_2 N_{\text{sent}}^N + w_3 N_{\text{drop}}^H + w_4 N_{\text{drop}}^N + w_5 N_{\text{to}}^H + w_6 N_{\text{to}}^N + w_7 (a_t^H)^2 + w_8 (a_t^N)^2 - w_9 \Delta e_t, \quad (4)$$

where N_{sent}^H and N_{sent}^N represent confirmed deliveries ($w_1 = 25.0, w_2 = 5.0$); N_{drop}^H and N_{drop}^N penalize buffer overflow drops and timeout expirations ($w_3 = -40.0, w_4 = -8.0, w_5 = -80.0, w_6 = -10.0$); quadratic terms penalize waiting queue latencies ($w_7 = -40.0, w_8 = -2.0$); and Δe_t penalizes energy depletion ($w_9 = 250.0$). If the scheduler selects sleep (Action 3), it receives a conservation bonus $r_{\text{sleep}} = +0.5$.

To ensure the agent reliably services critical traffic without eliminating operational flexibility, we introduce the soft policy guidance term:

$$r_t^{\text{guide}} = \begin{cases} +\alpha, & a_t = 0 \text{ and } q_t^H > 0 \\ -\beta, & a_t \neq 0, q_t^H > 0, a_t^H > \tau_u \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

with bonus $\alpha = 6.0$, penalty $\beta = -8.0$, and urgency age threshold $\tau_u = 0.55$. Under this formulation, the agent receives positive reinforcement for servicing HP packets. Critically, the omission penalty β is withheld if the waiting HP packet is young ($a_t^H \leq \tau_u$). This design grants the scheduler autonomy to temporarily withhold a newly arrived HP packet to flush an aging normal queue, intervening with negative feedback only when an HP packet approaches critical delay boundaries.

4 Methodology

4.1 Learning Architecture

We implement a Double Deep Q-Network (Double-DQN) architecture to eliminate the positive maximization bias inherent in standard Q-learning. In Double-DQN, greedy action selection is driven by the online parameters θ_t , whereas

target evaluation is computed using the slowly updated target parameters θ_t^- :

$$Y_t^{\text{DoubleQ}} = r_{t+1} + \gamma Q(\mathbf{s}_{t+1}, \arg \max_a Q(\mathbf{s}_{t+1}, a; \theta_t); \theta_t^-). \quad (6)$$

To accelerate sample convergence over rare, critical queue transitions, transitions are stored in a Prioritized Experience Replay (PER) buffer of capacity 100,000. Transitions are sampled with probability $P(i) = p_i^\mu / \sum_k p_k^\mu$, where $p_i = |\delta_i| + \epsilon_{\text{per}}$ represents the absolute temporal difference error and $\mu = 0.6$ regulates prioritization aggressiveness. Importance sampling weights $w_i = (N \cdot P(i))^{-\nu}$ correct for distribution distortion, where ν anneals linearly from 0.4 to 1.0 over 100,000 steps.

The deep Q-network consists of a fully connected Multi-Layer Perceptron (MLP) featuring an input layer matching $|\mathcal{S}| = 6$, two hidden layers each comprising 64 neurons activated by Rectified Linear Units (ReLU), and an output layer producing $Q(\mathbf{s}, \cdot)$ values across $|\mathcal{A}| = 5$. Target network weights are updated using soft Polyak averaging:

$$\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-, \quad (7)$$

with tracking parameter $\tau = 0.005$. Optimization is handled by Adam with an initial learning rate of 1×10^{-4} annealed via a Cosine Annealing schedule over 50 epochs. Gradients are clipped to a maximum L_2 norm of 1.0. Exploration follows an ϵ -greedy schedule decaying from $\epsilon = 1.0$ to $\epsilon = 0.05$ with multiplicative factor 0.99995 after 4000 warm-up steps. During evaluation, ϵ is set to 0.0 to enforce deterministic execution.

4.2 Baseline Schedulers

We benchmark the proposed policy-guided DQN against two baseline scheduling schemes:

- **Strict Priority (SP):** A deterministic, non-preemptive rule-based scheduler that unconditionally transmits high-priority packets whenever the HP queue is non-empty ($q_t^H > 0$). When $q_t^H = 0$, it transmits normal packets ($q_t^N > 0$). If both queues are empty, the radio transitions to the sleep state.
- **Random:** A stochastic baseline that samples actions uniformly at random from $\mathcal{A}$ at each decision step, representing the unguided performance floor.

4.3 Training and Validation Protocol

The learning pipeline runs for 50 epochs, with each epoch consisting of 5000 simulation steps. To ensure robust model selection, checkpoint saving is governed by a validation score evaluated across five fixed validation seeds (99991 to 99995). The scalar validation metric is defined as:

$$S_{\text{val}} = 0.40 \left(\frac{\text{PDR}}{100} \right) + 0.10 \left(1 - \frac{\text{PLR}}{100} \right) + 0.35 \left(\frac{\text{HR}}{100} \right) + 0.15 \left(1 - \min \left(\frac{D_H}{1.0}, 1.0 \right) \right), \quad (8)$$

Table 1. Simulation and Training Hyperparameters (V76 Configuration).

Parameter Category	Configured Value
Network Simulator	ns-3.40 interfaced via ns3-gym
Network Topology	100 Sensor Nodes + 1 Central Sink (150 m × 150 m)
Physical / MAC Layer	IEEE 802.15.4 (2.4 GHz ISM Band, CSMA/CA)
Control Interval (Δt)	100 ms per decision slot
Episode Duration	5000 decision steps (500 s real time)
Traffic Generation	Bursty Poisson (HP Alerts) + Periodic Telemetry (NP)
Queue Capacities	HP Buffer: 50 packets; Normal Buffer: 100 packets
State / Action Dimensionality	$ \mathcal{S} = 6$ continuous, $ \mathcal{A} = 5$ discrete
Neural Architecture	MLP: $6 \rightarrow 64 \rightarrow 64 \rightarrow 5$ (ReLU activations)
Replay Buffer Capacity	100,000 transitions (Prioritized: $\mu = 0.6, \nu = 0.4 \rightarrow 1.0$)
Mini-batch Size	128 transitions
Discount Factor (γ)	0.99
Optimizer / Learning Rate	Adam ($\text{lr} = 1 \times 10^{-4}$, Cosine Annealing)
Target Network Update	Soft Polyak update ($\tau = 0.005$)
Exploration Schedule	ϵ -greedy: $1.0 \rightarrow 0.05$ (decay factor 0.99995)
Evaluation Repetitions	20 matched runs per policy (seeds 12545–12564)

where HR is the HP delivery ratio percentage and D_H is the average HP latency in seconds. If HP packets are generated during validation but zero are delivered (HR = 0), a severe penalty of -0.25 is subtracted. Early stopping halts training if the validation score fails to improve for 15 consecutive epochs.

The final evaluation protocol executes 20 independent evaluation runs per policy across matching seeds (12545 to 12564). All comparative schedulers experience identical channel fading, node mobility, and traffic injection instances. Table 1 summarizes the simulation and hyperparameter settings.

4.4 Statistical Evaluation Protocol

Evaluated metrics include aggregate throughput (kbps), Packet Delivery Ratio (PDR %), Packet Loss Ratio (PLR %), average normal packet delay (s), average HP packet delay (s), specific energy consumption per delivered bit (nJ/bit), and the HP delivery ratio (%). All metrics are reported as the sample mean alongside their 95% confidence intervals (CI) computed using Student’s t -distribution.

For pairwise significance testing between the proposed DQN scheduler and Strict Priority across the 20 matched trials, we first verify the normality of paired differences via the Shapiro-Wilk test at $\alpha = 0.05$. If the paired differences conform to a normal distribution, we conduct a two-tailed paired Student’s t -test. If normality is rejected, we employ the non-parametric Wilcoxon signed-rank test. Practical effect sizes are characterized using paired Cohen’s d_z :

$$d_z = \frac{\bar{D}}{\sigma_D}, \quad (9)$$

where $\bar{D}$ and σ_D represent the sample mean and standard deviation of the paired differences across the 20 matched runs.

Table 2. Overall WSN Performance Metrics Across 20 Matched Evaluation Runs (Mean $\pm$ 95% CI).

Policy	Throughput (kbps)	PDR (%)	PLR (%)	Normal Delay (s)	HP Delay (s)	Energy (nJ/bit)	HP Delivery Ratio (%)
Proposed DQN-Edge	7.0267 $\pm$ 0.0702	50.3291 $\pm$ 0.6022	40.3309 $\pm$ 0.8373	0.4287 $\pm$ 0.0261	0.1431 $\pm$ 0.0026	443.8708 $\pm$ 16.9422	95.7779 $\pm$ 0.2916
Strict Priority (SP)	7.9104 $\pm$ 0.0054	56.6625 $\pm$ 0.3137	34.7224 $\pm$ 0.5326	0.6171 $\pm$ 0.0384	0.0274 $\pm$ 0.0015	417.0370 $\pm$ 17.2341	98.3374 $\pm$ 0.1941
Random Baseline	2.8000 $\pm$ 0.0350	20.0709 $\pm$ 0.3314	71.4943 $\pm$ 0.5450	0.8545 $\pm$ 0.0597	0.3521 $\pm$ 0.0129	806.7023 $\pm$ 22.7489	34.4862 $\pm$ 0.8563

5 Results

Table 2 reports the quantitative performance metrics compiled over the 20 seed-synchronized evaluation runs. Table 3 provides the corresponding matched statistical significance tests comparing the proposed DQN scheduler against the Strict Priority heuristic.

Table 3. Matched Pairwise Statistical Comparison: Proposed DQN-Edge vs. Strict Priority ($n = 20$ runs).

Evaluated Metric	Statistical Test	p -value	Paired Cohen's d_z
Throughput (kbps)	Paired t -test	2.656×10^{-16}	-5.8046
PDR (%)	Paired t -test	1.655×10^{-15}	-5.2568
PLR (%)	Paired t -test	4.749×10^{-13}	+3.8504
Avg. Normal Delay (s)	Wilcoxon signed-rank	1.907×10^{-6}	-3.3284
Avg. HP Delay (s)	Paired t -test	3.038×10^{-26}	+19.6001
Energy (nJ/bit)	Paired t -test	1.687×10^{-14}	+4.6310
HP Delivery Ratio (%)	Paired t -test	2.176×10^{-11}	-3.1005

5.1 Convergence and Training Stability

Figure 1 illustrates the training progression over 30 elapsed epochs prior to early stopping. The validation score trajectory shows rapid initial policy improvement over the first 5 epochs, transitioning into stable, monotonic refinement without policy collapse or loss divergence. The exploration rate ϵ decays steadily, confirming that the agent transitions from exploratory state-space coverage to stable deterministic execution. The model achieving the highest validation score at Epoch 12 was preserved for full benchmarking.

5.2 Comparative Analysis Against Heuristic Baselines

The empirical results establish that the policy-guided DQN scheduler substantially outperforms the unguided Random baseline across all performance metrics. Compared to Random, DQN increases aggregate throughput from 2.8000 kbps to 7.0267 kbps (a 150.9% gain), elevates overall PDR from 20.07% to 50.33%, lowers average HP latency from 0.3521 s to 0.1431 s (a 59.4% reduction), and improves energy efficiency from 806.70 nJ/bit to 443.87 nJ/bit (a 45.0% energy reduction).

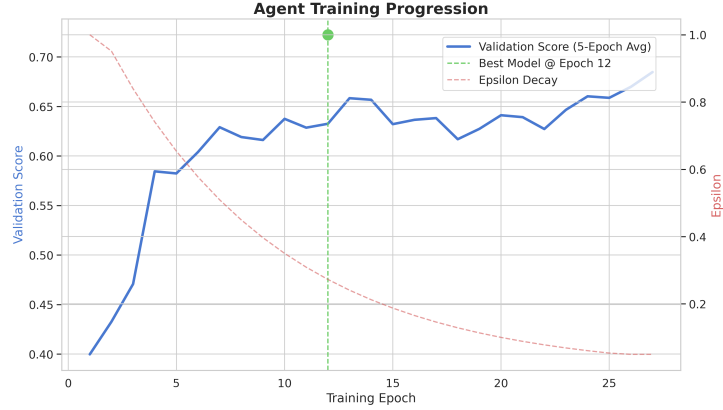


Fig. 1. Double-DQN training progression and validation-score trend over 30 epochs (V76).

Furthermore, the HP delivery ratio surges from 34.49% under random selection to 95.78% under the learned controller.

When compared against the dedicated Strict Priority baseline, however, the learned agent does not achieve superior throughput or HP delay. Strict Priority delivers an average throughput of 7.9104 kbps, an overall PDR of 56.66%, and an ultra-low HP latency of 0.0274 s alongside an HP delivery ratio of 98.34%. The paired statistical tests in Table 3 confirm that these margins are statistically significant ($p < 10^{-11}$), exhibiting large effect sizes.

5.3 Delay Distribution and Buffer Dynamics

Figure 2 presents a visual comparison of means and 95% confidence intervals across key QoS indicators, while Figure 3 details the empirical Cumulative Distribution Functions (CDFs) of packet latencies for both traffic classes.

The latency CDF curves in Figure 3 reveal the underlying mechanics of each scheduling policy. Strict Priority drives the HP delay curve aggressively toward the origin, ensuring that over 90% of all high-priority packets are cleared within 50 ms. However, this aggressive service discipline severely degrades normal telemetry traffic; the normal delay distribution for Strict Priority exhibits a long tail, with average latency climbing to 0.6171 s. In contrast, the proposed DQN agent balances queue service: it accepts a modest shift in HP delay (averaging 0.1431 s) to pull the normal traffic delay curve substantially to the left. As reflected in Table 2, DQN reduces average normal packet delay to 0.4287 s—a 30.5% latency reduction compared to Strict Priority ($p = 1.907 \times 10^{-6}$, $d_z = -3.3284$).

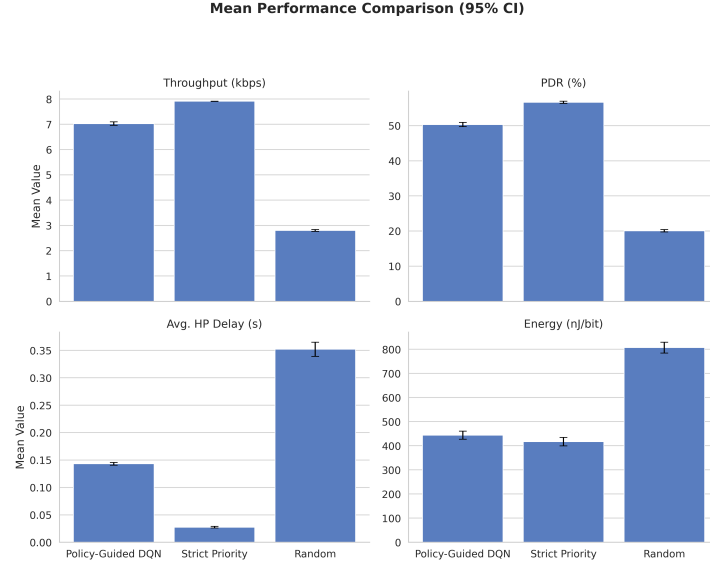


Fig. 2. Pairwise performance comparison (Mean $\pm$ 95% Confidence Interval) across key QoS metrics.

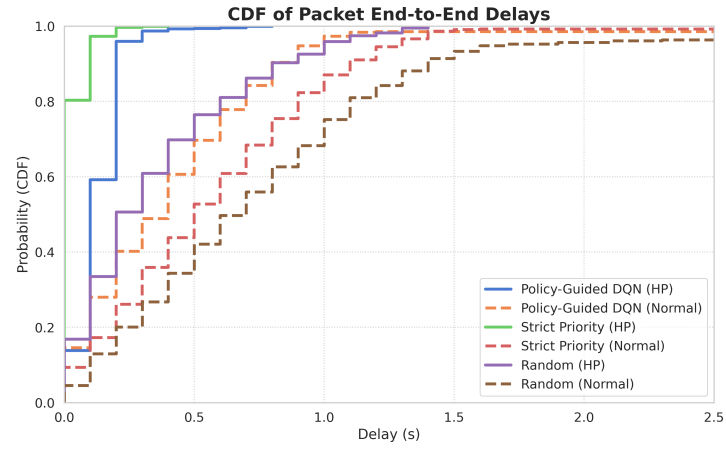


Fig. 3. Cumulative Distribution Function (CDF) of packet end-to-end delays for high-priority and normal-priority traffic classes.

5.4 Trade-Off Analysis in Smart Grid Telemetry

The quantitative divergence between DQN-Edge and Strict Priority demonstrates a clear engineering trade-off. Strict Priority behaves as a greedy, myopic heuristic; it prioritizes immediate HP transmissions regardless of normal queue status. In smart utility applications characterized by bursty fault activity, this myopic behavior induces buffer overflow for routine measurements, depriving grid operators of continuous current, voltage, and thermal sensor readings.

The policy-guided DQN scheduler learns a cooperative balancing policy. By observing head-of-line packet ages (a_t^H, a_t^N) , the agent learns that newly arrived HP packets can tolerate brief queuing periods without violating their 1.0 s deadline. During these brief windows, the agent opportunistically clears pending normal telemetry packets before they expire. By accepting a 2.56% reduction in HP delivery ratio and a 115.7 ms increase in HP delay, the learned scheduler delivers a 30.5% improvement in normal packet latency, preserving telemetry flow while maintaining robust 95.78% delivery for critical protection alerts.

5.5 Diagnostic Analysis: Reward Shaping vs. Policy Collapse

Our development process provided an important lesson regarding reward formulation in communications DRL. In earlier experiments using an unconstrained reward function (V74 configuration), the scheduler suffered from complete policy collapse: the agent converged to a degenerate policy that selected Action 1 (send normal) in 100% of decision steps. Because normal packets were abundant, transmitting normal traffic provided steady rewards, whereas attempting HP transmissions incurred risk due to bursty arrivals and contention. In that failed run, zero HP packets were delivered. However, because the simulator contained a zero-division safeguard defaulting average delay to 0.0 when no packets were delivered, the output superficially reported “0.0000 s delay”—a deceptive artifact that masked complete high-priority starvation.

Introducing the soft policy guidance in V76, paired with an HP-aware validation scoring metric, successfully resolved this pathology. By penalizing the scheduler only when an HP packet aged past $\tau_u = 0.55$, the agent was steered away from trivial local optima without sacrificing its ability to dynamically manage queues. This highlights that reinforcement learning applied to communication protocols requires explicit per-class packet tracking, action distribution validation, and tail latency profiling rather than aggregate reward monitoring alone.

6 Discussion

The results confirm that Double-DQN can learn stable, multi-objective scheduling behaviors that enhance fairness in mixed-criticality sensor clusters. Nevertheless, the fact that Strict Priority remains superior on raw high-priority latency indicates structural limitations in the unconstrained DRL formulation. First, the 6-dimensional observation vector provides partial observability, omitting instantaneous wireless channel state information (CSI) and physical-layer

signal-to-noise ratios. Second, single-agent Q-learning struggles to match the extreme responsiveness of deterministic heuristics on boundary cases without explicit safety overrides.

To advance learning-based scheduling for smart grid deployments, future work should address three methodological directions:

- **Constrained and Safe RL:** Formulating the scheduling task as a Constrained Markov Decision Process (CMDP) solved via Lagrangian relaxation or action projection layers to guarantee strict bounds on HP latency.
- **Distributional Value Approximation:** Utilizing Distributional RL (such as QR-DQN or IQN) to model the full distribution of queuing delays rather than expected values, mitigating tail-latency outliers.
- **Out-of-Distribution Sensitivity Analysis:** Testing trained policies against variable node densities, shifting HP-to-normal traffic ratios, and channel interference profiles to evaluate generalization beyond fixed training topologies.

7 Threats to Validity

Several factors may influence the generalizability of our empirical findings. First, all evaluations were conducted within discrete-event simulation in ns-3.40. While ns-3 provides high-fidelity modeling of 802.15.4 CSMA/CA protocols, physical hardware deployments introduce non-idealities such as transceiver switching jitter, clock drift, and complex spatial multipath fading that simulation abstractions omit.

Second, the observed performance boundaries are tied to the evaluated traffic load (100 nodes, 100 ms intervals, specific deadline limits). Varying the ratio of high-priority to normal traffic or increasing node contention could shift the trade-off surface between Strict Priority and DRL. Finally, our experimental setup focuses on single-hop cluster-head scheduling; extending these controllers to multi-hop meshed forwarding topologies represents an open research challenge.

8 Conclusion

This study presented a reproducible, statistically grounded evaluation of deep reinforcement learning for priority-aware packet scheduling in wireless sensor networks supporting smart energy infrastructure. By coupling ns-3 with PyTorch via ns3-gym, we trained a policy-guided Double-DQN agent over mixed-criticality traffic workloads. Across 20 seed-synchronized evaluation runs, the learned scheduler substantially outperformed random baseline allocation and preserved a 95.78% high-priority delivery ratio, but did not match Strict Priority in raw high-priority latency or throughput. However, the learned policy demonstrated superior multi-objective queue balancing, achieving a 30.5% reduction in normal telemetry delay and mitigating buffer starvation. These findings provide an open empirical benchmark that clarifies the practical performance boundaries and trade-offs of learning-based controllers in safety-critical sensor communications.

Reproducibility Package

To facilitate independent verification, the complete simulation artifacts—including the raw evaluation dataset (`raw_results_v76.csv`), training history (`training_log_v76.csv`), statistical summary tables (`final_results_v76.xlsx`), and the underlying ns-3 C++ and PyTorch source code—are made available for academic reuse.

References

1. Barker, A., Swamy, M.: Distributed cooperative reinforcement learning for wireless sensor network routing. In: 2022 IEEE Wireless Communications and Networking Conference (WCNC). pp. 2565–2570 (2022). <https://doi.org/10.1109/WCNC51071.2022.9771591>
2. Boni, A.K.C.S., Hassan, H., Drira, K.: Task offloading in autonomous iot systems using deep reinforcement learning and ns3-gym. In: Proceedings of the 11th International Conference on the Internet of Things. pp. 17–24 (2021). <https://doi.org/10.1145/3494322.3494325>
3. Dutta, H., Biswas, S.: Distributed reinforcement learning for scalable wireless medium access in iots and sensor networks. *Computer Networks* **202**, 108662 (2022). <https://doi.org/10.1016/j.comnet.2021.108662>
4. Frikha, M.S., Gammar, S.M., Lahmadi, A., Andrey, L.: Reinforcement and deep reinforcement learning for wireless internet of things: A survey. *Computer Communications* **178**, 98–113 (2021). <https://doi.org/10.1016/j.comcom.2021.07.014>
5. Gawlowicz, P., Zubow, A.: ns-3 meets openai gym: The playground for machine learning in networking research. In: Proceedings of the ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM) (2019)
6. Kim, H., Kim, Y.J., Kim, W.T.: Deep reinforcement learning-based adaptive scheduling for wireless time-sensitive networking. *Sensors* **24**(16), 5281 (2024). <https://doi.org/10.3390/s24165281>
7. Leong, A.S., Ramaswamy, A., Quevedo, D.E., Karl, H., Shi, L.: Deep reinforcement learning for wireless sensor scheduling in cyber-physical systems. *Automatica* **113**, 108759 (2020). <https://doi.org/10.1016/j.automatica.2019.108759>
8. Luong, N.C., Hoang, D.T., Gong, S., Niyato, D., Wang, P., Liang, Y.C., Kim, D.I.: Applications of deep reinforcement learning in communications and networking: A survey. *IEEE Communications Surveys & Tutorials* **21**(4), 3133–3174 (2019). <https://doi.org/10.1109/COMST.2019.2916583>
9. Mishra, S., Liang, J.M.: Dynamic sleep scheduling for wireless sensor transmissions based on reinforcement learning. *IEEE Sensors Letters* **7**(11), 1–4 (2023). <https://doi.org/10.1109/LSENS.2023.3327002>
10. Nagib, A.M., Abou-Zeid, H., Hassanein, H.S.: Toward safe and accelerated deep reinforcement learning for next-generation wireless networks. *IEEE Network* **37**(2), 182–189 (2023). <https://doi.org/10.1109/MNET.106.2100578>
11. Sharma, A., Chauhan, S.: A distributed reinforcement learning based sensor node scheduling algorithm for coverage and connectivity maintenance in wireless sensor network. *Wireless Networks* **26**(6), 4411–4429 (2020). <https://doi.org/10.1007/s11276-020-02350-y>